Table Methods

Students learn about *table methods* , which allow them to order, filter, and build columns to extend the animals table.

Lesson Goals	Students will be able to… • order the Animals Dataset by a number of criteria • filter the Animals Dataset by species, fixed status, and age • add a column to the Animals Dataset
Student-facing Lesson Goals	• Let's learn how transform one table into another.
Prerequisites	• Simple Data Types • Contracts • Functions: Contracts, Examples & Definitions • Bar and Pie Charts
Materials	• Lesson Slides • Table Methods Starter File • *Handout: Function Cards* • Reading Row and Function Definitions (Page 27) • Exploring Methods (Page 28) • *Optional: What Table Do We Get?* • **Classroom visual:** Language Table

Glossary

Boolean :: a type of data with two values: true and false

percentage :: a ratio showing the parts per hundred

Functions that Work on Rows *20 minutes*

Overview

Students review row definitions, and get more practice reading the definitions of functions that work on rows.

Launch

Load the [Table Methods Starter File](), go to the File menu, and click "Save a Copy".

Explain to students that this activity is about reading carefully and getting more comfortable with functions that work on rows.

Investigate

With your partner, complete items 1 through 8 on [Reading Row and Function Definitions]().

Direct students to take a look at the three examples for `is-dog`. Each one shows us a different way of thinking about examples, in this case using a row that should return `false`:

```
examples:
  is-dog(cat-row) is false
  is-dog(cat-row) is "cat" == "dog"
  is-dog(cat-row) is cat-row["species"] == "dog"
end
```

1. The first example just tells us the answer we think we'll get back. `cat-row` is NOT a dog, so we expect `false`!
2. The second example shows us some of the work involved: we know the species of the row is `"cat"`, and comparing that to the String `"dog"` will return `false`.
3. The third example shows *all the work*: given the `cat-row`, we lookup the value in the `"species"` column and compare it to the String `"dog"`.

- Add three *true* examples for `is-dog`, this time using the `dog-row` you defined above.
- Complete the worksheet.

Synthesize

- Who can explain what `is-old` does?
  - *It looks up the age of the row, and asks if its greater than 10.*
- Who can explain what `lookup-name` does?
  - *It looks up the value in the `name` column.*
- Who can explain what `lookup-fixed` does?
  - *It looks up the value in the fixed column.*
- Who can explain what `label` does?
  - *It prints the animal's name in red.*

- Who can explain what `kilos` does?
    - *It divides the pounds by 2.2.*

Challenge students to use terminology like "looks up the value in the X column" when describing a lookup.

Ordering, Filtering, and Building *30 minutes*

Overview

Students learn to sort rows of a table in ascending or descending order, filter the rows using a Boolean-producing function, and build a new column.

Launch

Select 6-8 students, each of whom will represent a unique Row of a dataset called "Students." Arrange them in a line at the front of the room.

Explain to students that: * Each of you represents a Row in a table of students * Standing together, you represent a *sample* of the class, stored in a table. * Like you, each student has columns like "First Name", "Last Name", "Birthday", "Favorite Food", etc.

A Table is a collection of Rows. And we can *do things* to collections! One thing we can do is **order the rows** by some criteria. What are some ways to sort a table of students?

Tell the class that you are going to select a volunteer who will play the role of… **order-by**! Announce to your volunteer "Sort the Rows by height, from smallest to largest!", and have them do it. Then announce to your volunteer "Sort the Rows by height, from largest to smallest!", and have them do it.

Solicit other ideas, like sorting by birthday. Try sorting in ascending **and** descending order, using language appropriate cues: "order-by 'birthday', ascending equals true", or "order-by height, ascending equals false!" After a few iterations, give the volunteer a round of applause!

Another thing we can do is **filter the rows** by some criteria. In our animals dataset, for example, we might want to filter the table to show only the rows for dogs, or only the rows for cats older than 5. What are some ways to filter our table of students?

Tell the class that you are going to select a volunteer who will play the role of… **filter**! Hand the volunteer a function card from the Function Cards set. (You may need to choose your students strategically. The demonstration that follows, for instance, requires at least one student who wears glasses.) Instruct Filter to read the card carefully, making sure they understand what it will do for each "Row" (student!) in the "Table" (line of students), but to keep that information a secret from the other students.

Explain to the class that the volunteer playing Filter will walk from one student to the next, referring to the function (on the card) in order to determine if each student should step forward or step backward.

Here's how that might look if Filter chose a function card `has-glasses`.

- *Filter stands in front of Student 1 and checks if they are wearing glasses.*
- **Filter to Student 1 (who wears glasses)**: Step forward. (Student 1 steps forward.)
- *Filter stands in front of Student 2 and checks if they are wearing glasses.*
- **Filter to Student 2 (who does not wear glasses)**: Step back. (Student 2 steps back.)

Have your Filter volunteer go through all their peers, applying their card to each one. Based on who stepped forward and backward, what was on the card? Repeat for several cards.

Investigate

Pyret Tables have their own methods for sorting, filtering, and more. The exploration that students will complete is divided into three sections, accordingly.

Complete the "Ordering Rows" section on Exploring Methods to discover how the `.order-by` method works.

After students have completed the section, lead a discussion to confirm that students understand the following key ideas:

- `.order-by` consumes a String (the name of the column by which to sort) and a ***Boolean*** (`true` for ascending, `false` for descending), and sorts the rows according to that column.
- The `.order-by` method produces a *new* table rather than changing the existing one.
- How could we test that a new table was produced? We could sort the table, then evaluate `animals-table` and see if it stayed sorted.

- Complete the "Filtering Rows" section on Exploring Methods
- Find the contract for `.filter` in your Contracts page.

Confirm that students understand the following key ideas:

- The Domain of `.filter` is a *Boolean-producing function*
- The `.filter` method works by producing a new table containing only rows for which the function returns `true` .
- The `.filter` method creates a new table.

Complete "Building Columns", the last section of Exploring Methods

Confirm that students understand the following:

- `.build-column` takes in a String and a *function*
- The `.build-column` method produces a new table with an extra column, using the String for the column title, and fills in the values by applying the function to every Row.
- Building a column in the `animals-table` produces a *new* table.

Want some more practice? Complete What Table Do We Get?.

Common Misconceptions

Students often think that these methods *change* the table! In Pyret, all table methods produce a *brand new table* . If we want to save that table, we need to define it. For example: `cats = animals-table.filter(is-cat)` .

Synthesize

Being able to define functions and use them with Table Methods is a *huge* upgrade in our ability to analyze data!

- Suppose we wanted to determine whether cats or dogs get adopted faster. How might using the `.filter` method help?
- If the shelter is purchasing food for older cats, what filter would we write to determine how many cats to buy for?
- A dataset from Europe might list everything in metric (centimeters, kilograms, etc), so we could build a column to convert that to imperial units (inches, pounds, etc).
- A dataset about schools might include columns for how many students are in the school and how many of those students identify as multi-racial. But when comparing schools of different sizes, what we really want is a column showing what ***percentage*** of students identify as multi-racial. We could use `.build-column` to compute that for every row in the table.
- **What are some ways you might want to filter *your* dataset?**
- **What are some columns you might want to build for *your* dataset?**